



DOSSIER PROJET

Thomas Bruston

Titre Professionnel DWWM — Niveau 5
Organisme de formation Studi

Session juin-juillet 2026



<https://vitegourmand.tech>

TABLE DES MATIERES

Table des matières

Remerciements

Introduction

Résumé du projet et compétences REAC couvertes

1. Présentation du projet

- 1.1 cahier des charges
- 1.2 Règles métier principales
- 1.3 Diagramme d'utilisation

2. Gestion de projet

- 2.1 Méthode et planning
- 2.2 Outils de gestion

3. Développer la partie front-end

- 3.1 Charte graphique
- 3.2 Wireframes et mockups
- 3.3 Interfaces statiques (HTML/CSS/RGAA)
- 3.4 Accessibilité RGAA
- 3.5 Interfaces dynamiques (AJAX, JS vanilla)
- 3.6 Sécurité front (CSRF, XSS)

4. Développer la partie back-end

- 4.1 Base de données (MCD, SQL)
- 4.2 Architecture MVC PHP POO
- 4.3 Composants d'accès aux données (PDO + MongoDB)
- 4.4 Composants métier (Services, Entités)
- 4.5 Sécurité back-end, protection des données et conformité RGPD

5. Tests et jeu d'essai

- 5.1 tests unitaire PHP Unit
- 5.2 jeu d'essai

6. Veille technologique et sécurité

7. Déploiement

- 7.1 Conteneurisation avec Docker
- 7.2 Difference entre développement et production
- 7.3 Sécurité du déploiement
- 7.4 Procédure de déploiement sur VPS

8. Conclusion

9. Annexes

Remerciements

Je tiens à remercier l'équipe pédagogique Studi sans laquelle je n'aurai pas pu réaliser ce projet.

Je tiens également à remercier mes formateurs Nathan Derhore et Christian Lohez pour leur soutien et leurs apports de connaissances tout au long de mon travail sur ce projet.

Je souhaiterais remercier ceux qui m'ont donné envie de devenir développeur web et qui m'ont accompagné dans ce but, Nicolas Rocchi et Dimitri Constant qui m'ont permis d'effectuer des immersions professionnelles dans leurs entreprise respectives, d'y découvrir la réalité du métier de développeur web et ses contraintes quotidiennes, et qui m'ont soutenu lors de la réalisation de ce projet, ainsi que Medhi Callemeyn pour son aide précieuse lors du développement de la partie back-end du projet.

Je tiens enfin à remercier les autres apprenants de ma formation, avec qui nous avons pu partager nos difficultés et les solutions qui les résolvent.

Introduction

Après plusieurs années dans le graphisme et le management, j'ai fait le choix de me reconverter dans le développement web, un domaine qui m'attirait depuis longtemps et dans lequel je souhaitais m'investir.

C'est dans cette optique que j'ai intégré la formation Développeur Web et Web Mobile de Studi, qui m'a permis d'acquérir les compétences nécessaires à la conception et au développement d'applications web complètes.

Au préalable, j'ai eu l'opportunité d'effectuer deux immersions professionnelles particulièrement enrichissantes. La première chez AIRDDM, une agence spécialisée en data driven marketing, où j'ai pu découvrir les contraintes et les exigences du développement web en environnement professionnel. La seconde chez Tachycard, une startup développant une application web à destination des cardiologues et des patients équipés de pacemakers, où j'ai pu appréhender les enjeux techniques d'une application critique nécessitant fiabilité et sécurité.

C'est fort de ces expériences que j'ai développé **Vite & Gourmand**, une application web de commande de menus traiteur pour une entreprise bordelaise. Ce projet, réalisé en autonomie sur une période de quatre mois, constitue le projet de validation de mon Titre Professionnel DWWM Niveau 5. Il m'a permis de mettre en œuvre l'ensemble des compétences acquises lors de ma formation, de la conception des interfaces utilisateur au déploiement de l'application, en passant par le développement d'une architecture back-end sécurisée en PHP orienté objet, la gestion de bases de données relationnelles et non relationnelles, ainsi que la conteneurisation via Docker.

Ce dossier de projet retrace l'ensemble de la démarche de conception et de développement de cette application, en détaillant les choix techniques effectués, les difficultés rencontrées et les solutions mises en œuvre pour y répondre.

Résumé du projet

Dans le cadre de mon Titre Professionnel Développeur Web et Web Mobile Niveau 5, je réalise une application web et web-mobile pour une entreprise traiteur bordelaise, Vite & Gourmand.

L'entreprise a besoin d'une application permettant à ses clients de commander des menus en ligne et de bénéficier d'un service de livraison à domicile. Elle a également besoin de pouvoir gérer son activité en interne, en modifiant les menus, en gérant les employés et en obtenant des données d'analyse sur son chiffre d'affaires.

Mon rôle est de développer la totalité de l'application, de sa conception graphique à son déploiement en production, en passant par sa logique métier. Travaillant seul sur ce projet, je mets en œuvre les compétences acquises lors de ma formation ainsi que les ressources citées dans les remerciements.

Compétences REAC couvertes :

Activité Type 1 : Développer la partie front-end d'une application web sécurisée

- CP1 : Installer et configurer son environnement de travail
- CP2 : Maquetter une application web ou web mobile
- CP3 : Réaliser des interfaces utilisateur statiques et adaptables
- CP4 : Développer la partie dynamique des interfaces utilisateur

Activité Type 2 : Développer la partie back-end d'une application web sécurisée

- CP5 : Créer une base de données relationnelle
- CP6 : Développer les composants d'accès aux données SQL et NoSQL
- CP7 : Développer des composants métier côté serveur
- CP8 : Documenter le déploiement d'une application web

1. Présentation du projet

Cahier des charges

Ce projet est un projet « starting from scratch » le cahier des charges est donc bien complet.

L'Utilisateur doit arriver en premier lieu sur une page d'accueil comprenant la présentation de l'entreprise et de son équipe et les avis déposés par les clients.

Il doit pouvoir s'y authentifier ou s'y inscrire.

Elle doit intégrer un menu de navigation comprenant un retour vers l'accueil, un accès au menus, un lien permettant la connexion et un autre vers une page de contact.

L'utilisateur doit ensuite pouvoir avoir accès aux horaires, aux conditions générales de vente et aux mentions légales en pied de page.

Il doit avoir également accès aux menus et leurs différents détails qu'il doit pouvoir filtrer.

Il peut ainsi commander des menus et indiquer les informations relatives à la livraison de ceux-ci avant d'accéder à son panier.

Il peut ensuite accéder à sa commande et la modifier ou l'annuler.

Coté entreprise, le gérant souhaite la création d'un espace employé permettant à l'équipe de modifier la carte des menus, de valider les avis ou de les supprimer et de modifier les horaires d'ouverture du restaurant.

Les employés doivent également avoir accès aux commandes pouvant ainsi mettre à jour leur statut.

Il souhaite ensuite la mise en place d'un accès administrateur lui étant réservé afin d'obtenir les mêmes autorisations que les employés.

Il désire pouvoir gérer ses employés en en ajoutant ou en en supprimant et avoir accès à une analyse de données par le biais d'un graphique détaillant les commandes et le chiffre d'affaire qu'elles représentent .

Vite & Gourmand est une application web de commande de menus traiteur en ligne. Elle est accessible à quatre types d'utilisateurs aux droits progressifs.

Visiteur (non connecté)

- Consulter la page d'accueil avec présentation de l'entreprise et avis clients validés
- Parcourir les menus avec filtres dynamiques (prix, thème, régime, nombre de personnes)
- Consulter le détail d'un menu
- Accéder au formulaire de contact
- Créer un compte et se connecter
- Consulter les CGV, mentions légales et horaires

Utilisateur connecté

- Commander un menu (formulaire multi-étapes avec calcul de prix automatique)
- Consulter et suivre ses commandes (historique des statuts)
- Modifier ou annuler une commande si statut "en attente"
- Laisser un avis (note 1-5 + commentaire) sur une commande terminée
- Modifier ses informations personnelles

Employé

- Gérer les menus et les plats (CRUD complet)
- Mettre à jour les statuts de commande
- Annuler une commande avec motif obligatoire
- Valider ou refuser les avis clients
- Gérer les horaires d'ouverture

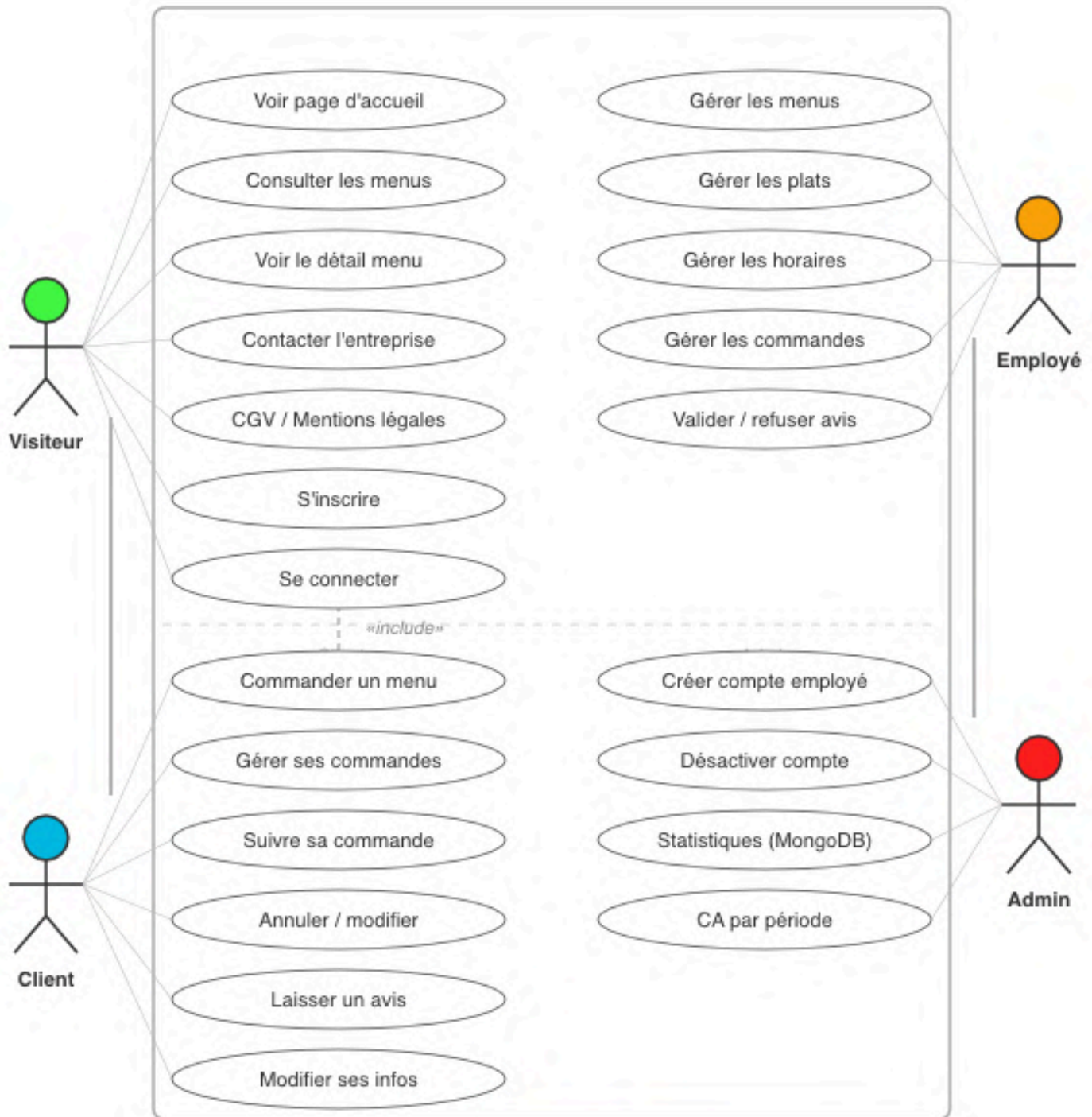
Administrateur

- Toutes les fonctionnalités employé
- Créer et désactiver des comptes employé
- Consulter les statistiques de commandes par menu (MongoDB)
- Visualiser le chiffre d'affaires par période (graphique)

Règles métier principales

Règle	Détail
Calcul du prix	Prix/personne × nombre de personnes
Réduction	-10% si nombre de personnes ≥ minimum + 5
Frais de livraison	+5€ + 0,59€/km hors Bordeaux
Statuts de commande	En attente → acceptée → en préparation → en cours de livraison → livrée → terminée //// annulée //// retour de matériel
Avis	Uniquement sur commande terminée
Annulation/ Modification	Uniquement si statut "en attente"

Diagramme d'utilisation



Héritage cumulatif des rôles :

Admin ▷ Employé
Client ▷ Visiteur

2. Gestion de projet

2.1 Méthode et planning

Ce projet a été développé en autonomie sur une période de quatre mois. J'ai organisé mon travail en suivant un ordre de développement planifié, de l'architecture de base jusqu'au déploiement :

- **Semaines 1-2** : Architecture MVC, Docker, base de données.
- **Semaines 3-4** : Authentification, pages statiques et charte graphique.
- **Semaines 5-6** : Menus publics et filtres AJAX.
- **Semaines 7-8** : Système de commande et calcul de prix.
- **Semaines 9-10** : Espace utilisateur (suivi commandes, avis).
- **Semaines 11-12** : Espaces employé et administrateur.
- **Semaines 13-14** : Sécurité, accessibilité RGAA, tests PHPUnit.
- **Semaines 15-16** : Déploiement, documentation et finalisation du dossier.

2.2 Outils de gestion

J'ai utilisé Trello pour organiser et suivre l'avancement des tâches sous forme de tableau Kanban.

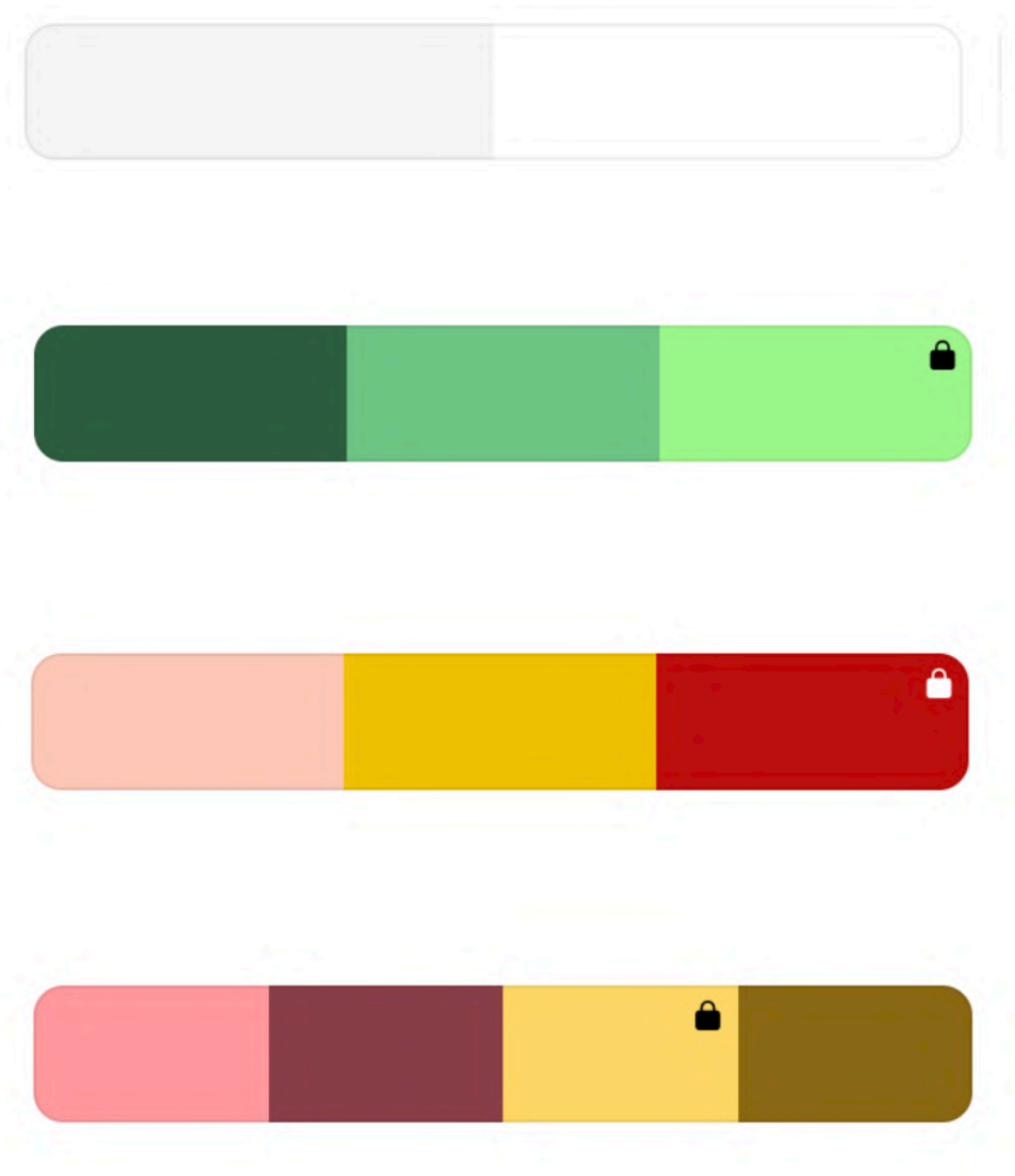
Git Flow a été mis en place pour le versionnage avec une branche **main** stable contenant le code de production, et une branche **develop** d'intégration sur laquelle l'ensemble des développements ont été réalisés et committés.

3.Développer la partie front-end

3.1 Charte graphique

La charte graphique de Vite & Gourmand s'inspire de l'identité visuelle de la gastronomie bordelaise — sobre, élégante et chaleureuse. Chaque choix de couleur et de typographie a été pensé pour refléter le positionnement haut de gamme de l'entreprise traiteur.

Palette de couleurs



Vert foncé #2d5f3f	Couleur principale : header, footer, boutons, menus
Vert clair #6ec585	Couleur secondaire : hover, badges, accents
Or #f0c103	Mise en valeur : prix, éléments importants
Rose #ff9a9e	Badges thèmes : gastronomie, bistronomie
Orange clair #FDC8B6	Badges régimes : contours formulaires
Fond #f5f5f5	Arrière-plan général des pages
Blanc #ffffff	Fonds de cartes, textes sur fond sombre

Le **vert foncé #2d5f3f** constitue la couleur identitaire de l'application. Il évoque à la fois la fraîcheur des produits locaux et l'écologie. Les tons chauds rose et orange apportent une touche de contraste.

Typographie

Deux polices de caractères ont été sélectionnées pour créer une hiérarchie visuelle claire :

- **Italiana** : police serif élégante utilisée pour les titres et le nom de l'application. Son style raffiné renforce le positionnement gastronomique haut de gamme.
- **Lucida Sans** : police sans-serif lisible utilisée pour le corps du texte, les boutons et les éléments de navigation. Elle garantit une bonne lisibilité sur tous les écrans.

Ces deux polices sont chargées via Google Fonts et définies comme variables CSS dans :root pour garantir leur cohérence sur l'ensemble de l'application.

Logo

Le logo de Vite & Gourmand représente une toque de chef entourée de lauriers, symboles de l'excellence culinaire. Il est décliné en couleur **Rose #ff9a9e** sur fond clair, et apparaît centré dans la barre de navigation sur desktop comme sur mobile.

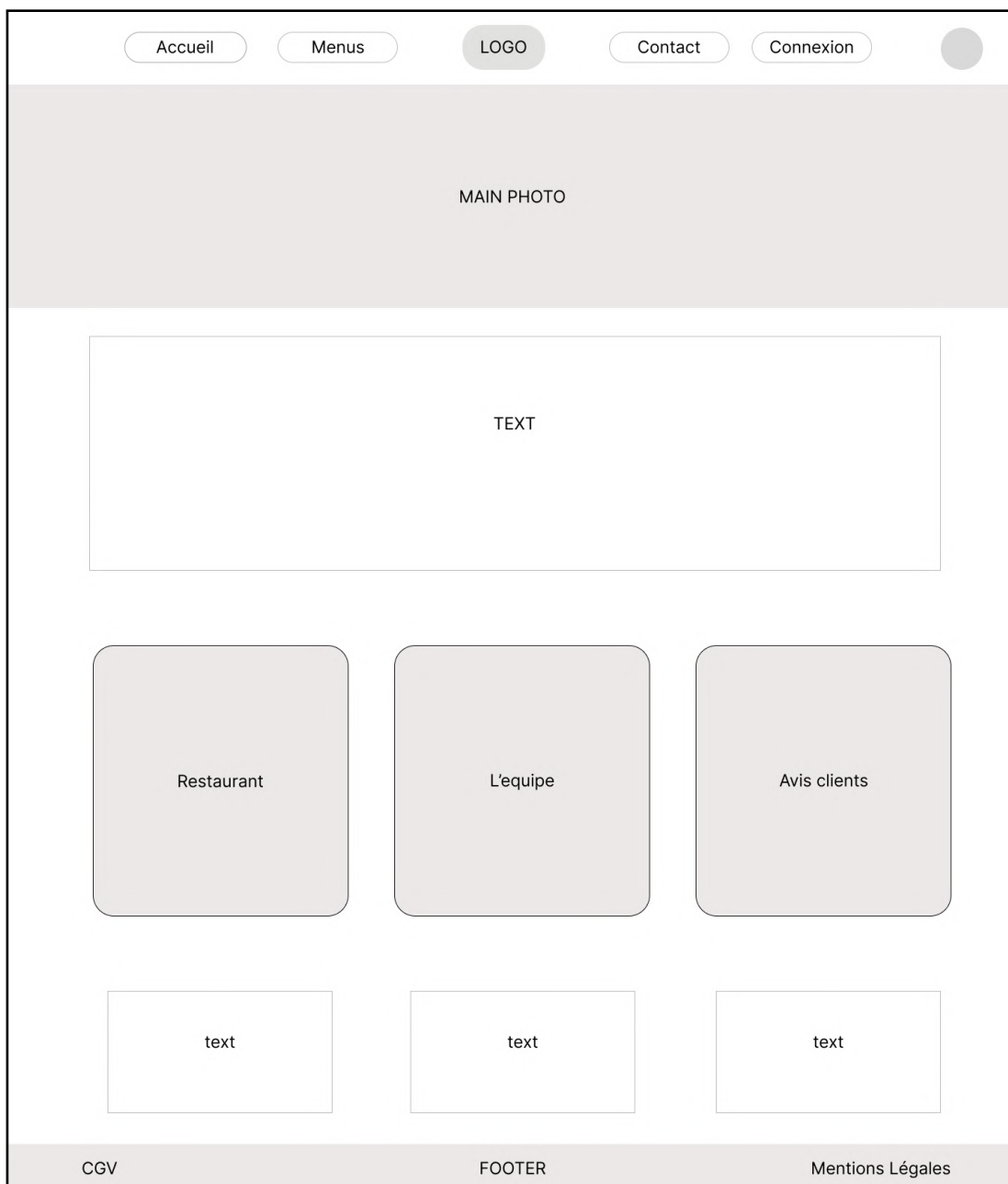
3.2 Wireframes et mockups

J'ai décidé de réaliser la totalité de l'application avec le logiciel Figma.

J'ai installé plusieurs bibliothèques de la communauté afin de réaliser les wireframes et les mockups de toute l'application afin de visualiser l'entièreté de l'expérience utilisateur desktop et mobile.

Ceci m'a permis d'avoir une vue d'ensemble de l'application mais aussi de gagner un temps précieux lors de son développement tout en conservant un fil conducteur.

Wireframe Desktop accueil



Wireframe Desktop connexion

Accueil

Menus

LOGO

Contact

Connexion

PHOTO

SE CONNECTER

E-mail

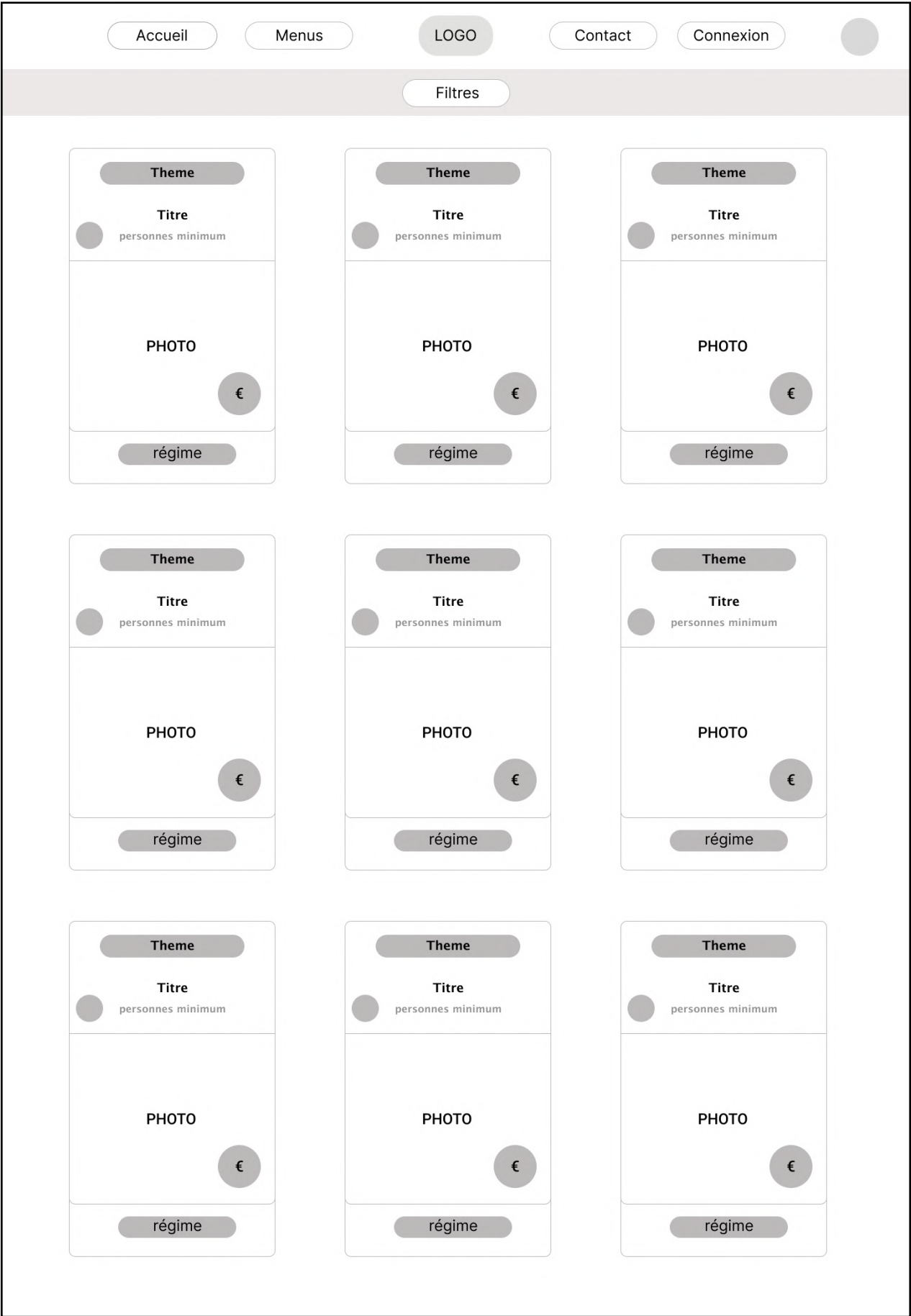
Mot de passe

Mot de passe oublié ?

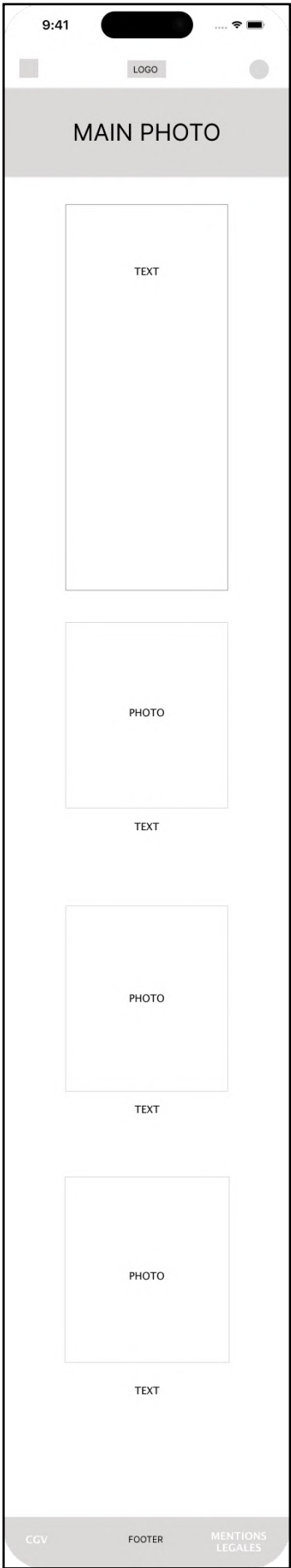
Pas encore inscrit(e)?

CONNEXION

Wireframe Desktop menus



Wireframe Mobile accueil



Wireframe Mobile connexion



9:41

LOGO

MAIN PHOTO

SE CONNECTER

Votre E-mail

Votre mot de passe

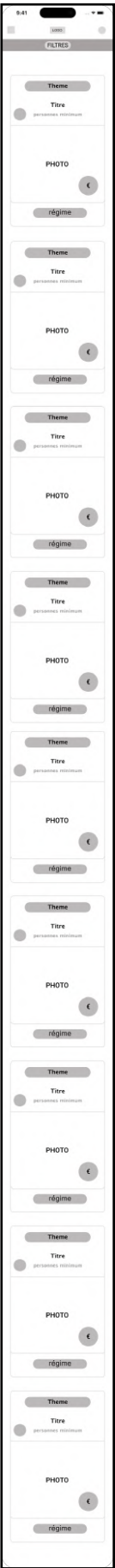
Mot de passe oublié ?

Pas encore inscrit ?

CONNEXION

Detailed description: This is a mobile app wireframe for a login screen. At the top, there is a status bar with the time '9:41', a black pill-shaped notch, and icons for signal strength, Wi-Fi, and battery. Below the status bar is a header area with a small square icon on the left, a 'LOGO' placeholder in the center, and a small circle icon on the right. The main content area has a light gray background. It features a large rectangular area labeled 'MAIN PHOTO'. Below this is a white rounded rectangle containing the login form. The form is titled 'SE CONNECTER'. It has two input fields: 'Votre E-mail' and 'Votre mot de passe'. The password field has a toggle icon (an eye) on the right. Below the password field are two links: 'Mot de passe oublié ?' and 'Pas encore inscrit ?'. At the bottom of the form is a dark gray button labeled 'CONNEXION'.

Wireframe Mobile menus





Mockup Desktop connexion

[Accueil](#)[Menus](#)[Contact](#)[Connexion](#)



SE CONNECTER

Votre E-mail

Votre mot de passe

[Mot de passe oublié ?](#)

[Pas encore inscrit ?](#)

CONNEXION

Mockup Desktop menus

Accueil

Menus



Contact

Connexion



Filtres

Gastronomie

"Le Réception"

 2 personnes minimum



36€

Régime : Classique

Bistronomie

"Le Bistrot"

 2 personnes minimum



25€

Régime : Classique

Bistronomie

"Le Dinatoire"

 6 personnes minimum



20€

Régime : Classique

Cuisine du monde

"L'Athènes"

 2 personnes minimum



25€

Régime : Classique

Cuisine du monde

"Le Marrakech"

 4 personnes minimum



28€

Régime : Sans porc

Cuisine du monde

"Le Barcelone"

 4 personnes minimum



25€

Régime : Classique

Cuisine du monde

"Le Roma"

 6 personnes minimum



25€

Régime : Classique

Végétarien

"Le Champêtre"

 2 personnes minimum



26€

Régime : Végan

Evenements

"Le St Valentin"

 2 personnes minimum



30€

Régime : Classique

Mockup Mobile accueil





The mockup shows a mobile app interface for login. At the top, the status bar displays the time 9:41, a black pill-shaped notch, and icons for cellular signal, Wi-Fi, and battery. Below the status bar is a navigation bar with a green hamburger menu icon on the left, a red circular logo with a white crown in the center, and a green circular profile icon on the right. The main content area features a large image of a gourmet dish on a white plate. Below the image is a white rounded rectangle with an orange border containing the login form. The form has a title 'SE CONNECTER', followed by labels 'Votre E-mail' and 'Votre mot de passe'. Each label is followed by a text input field with a placeholder 'Entrez votre E-mail' and 'Entrez votre mot de passe' respectively. The password field includes a toggle icon. Below the inputs are two links: 'Mot de passe oublié ?' and 'Pas encore inscrit ?'. At the bottom is a green button with the text 'CONNEXION'.

9:41

SE CONNECTER

Votre E-mail

Entrez votre E-mail

Votre mot de passe

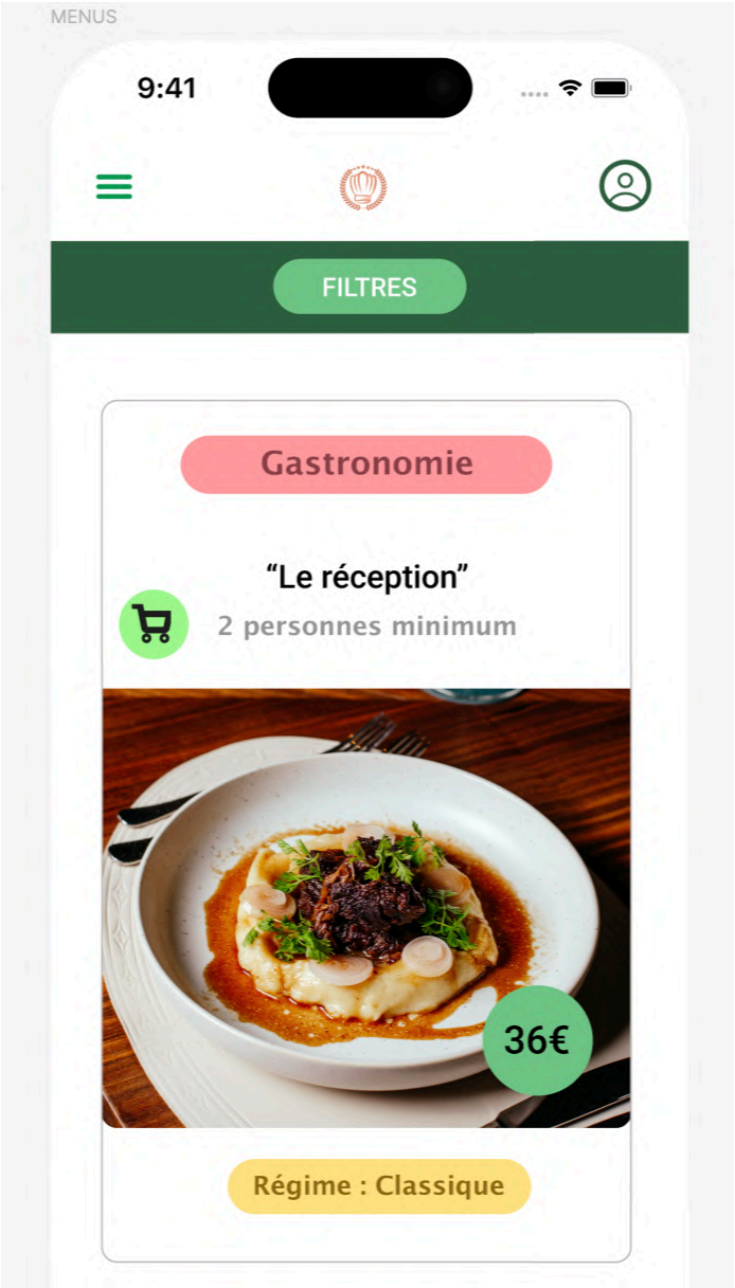
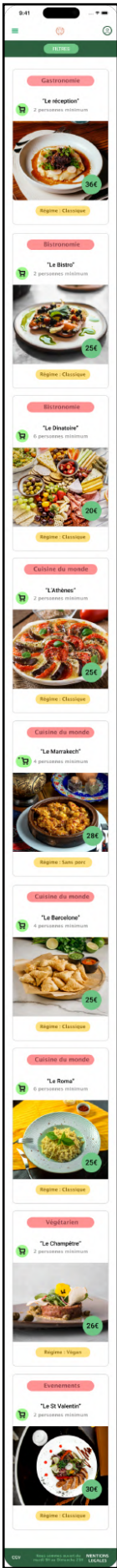
Entrez votre mot de passe

Mot de passe oublié ?

Pas encore inscrit ?

CONNEXION

Mockup Mobile menus



3.3 Interfaces statiques (HTML/CSS)

Pour développer l'architecture de l'application j'ai utilisé l'IDE Visual Studio Code avec les extensions LiveServer, HTML CSS Support, CodeRunner et IntelliSense for CSS class names in HTML.

Structure HTML sémantique

Chaque page respecte les balises sémantiques HTML5 imposées par le référentiel REAC et les normes RGAA : <header> pour l'en-tête de navigation, <main> pour le contenu principal, <nav> pour les menus de navigation, <section> pour les blocs de contenu et <footer> pour le pied de page. Cette structuration sémantique améliore l'accessibilité pour les lecteurs d'écran

Architecture CSS

Le CSS est organisé sans framework, en CSS vanilla pur. Un fichier main.css global est chargé sur toutes les pages et contient les variables CSS, les styles du header, du footer et des composants partagés. Chaque page dispose ensuite de son propre fichier CSS.

Les variables CSS définies dans root centralisent l'ensemble des valeurs de la charte graphique :

```
:root {
  --color-vert-fonce: #2d5f3f;
  --color-vert-clair: #6ec585;
  --color-or: #f0c103;
  --color-rose: #ff9a9e;
  --color-orange: #FDC8B6;
  --color-bg: #f5f5f5;
  --color-blanc: #ffffff;
  --color-noir: #000000;
  --font-italiana: 'Italiana', sans-serif;
  --font-lucida: 'Lucida Sans', 'Lucida Sans Regular',
    'Lucida Grande', 'Lucida Sans Unicode',
    Geneva, Verdana, sans-serif;
  --border-radius-btn: 30px;
  --transition: 0.3s ease;
  --shadow-card: 0 2px 8px rgba(0, 0, 0, 0.1);
  --shadow-hover: 0 4px 8px rgba(45, 95, 63, 0.3);
}
```

Cette approche garantit la cohérence visuelle sur l'ensemble de l'application et facilite la maintenance : modifier une couleur dans root la met à jour sur toutes les pages simultanément.

Responsive design

L'application adopte une approche desktop-first avec des media queries max-width pour adapter l'affichage aux écrans mobiles. Trois points de rupture sont définis : 1024px pour les tablettes, 780px et 480px pour les mobiles. Sur mobile le menu de navigation desktop est remplacé par un menu burger

Exemple:

```
@media (max-width: 480px) {  
  header { padding-right: 80px; }  
  .navbar { margin: 0; }  
  .header-btn { display: none; }  
  .burger { display: block; }
```

3.4 Accessibilité RGAA

L'accessibilité a été prise en compte dès la conception de l'application, en respectant les recommandations du Référentiel Général d'Amélioration de l'Accessibilité (RGAA)

Mesures mises en place

La structure sémantique HTML5 décrite en section 3.3 constitue la première couche d'accessibilité, les balises <header>, <main>, <nav>, <section> et <footer> permettent aux lecteurs d'écran de naviguer efficacement dans le contenu.

Les formulaires respectent l'association obligatoire entre chaque champ et son libellé via les attributs for et id.

Les champs de type date et heure disposent d'un attribut aria-label explicite pour les utilisateurs utilisant des technologies d'assistance :

```
<div class="form-group">  
  <label for="date">Date de prestation</label>  
  <input type="date"  
    id="date"  
    name="date_prestation"  
    aria-label="Date de prestation – format jour/mois/  
    année">
```

Les composants interactifs utilisent les attributs ARIA appropriés. Par exemple le bouton de suivi de commande expose son état et sa cible via `aria-expanded` et `aria-controls` :

```
<button type="button" class="btn btn-follow »  
                                aria-controls="followMenu-  
                                aria-expanded="false">  
                                SUIVRE  
</button>
```

Les contrastes de couleurs ont été vérifiés et corrigés pour respecter le ratio minimum de 4.5:1 imposé par les normes WCAG AA. Le texte des horaires dans le footer utilise le blanc #ffffff sur fond vert foncé #2d5f3f, produisant un ratio largement supérieur au minimum requis.

Résultats Lighthouse

Les audits d'accessibilité réalisés avec l'outil Lighthouse de Google Chrome donnent les scores suivants :

Page	Score Accessibility
Accueil	92/100
Connexion	99/100
Menus	98/100

Ces scores confirment la conformité de l'application aux normes d'accessibilité en vigueur. Les captures d'écran des rapports Lighthouse sont disponibles en annexe.

3.5 Interfaces dynamiques (AJAX, JS vanilla)

L'ensemble des interactions dynamiques de l'application est développé en JavaScript vanilla ES6+, sans bibliothèque externe, conformément aux contraintes du projet.

Filtres de menus sans rechargement de page

La page menus intègre un système de filtres dynamiques par thème, régime, nombre de personnes et prix maximum. Lorsque l'utilisateur valide ses filtres, une requête AJAX est envoyée au serveur sans rechargement de page via l'API `fetch()` :

```

function fetchMenus() {
    const params = new URLSearchParams();
    if (activeFilters.theme) params.append('theme',
activeFilters.theme);
    if (activeFilters.regime) params.append('regime',
activeFilters.regime);
    if (activeFilters.personnes) params.append('personnes',
activeFilters.personnes);
    if (activeFilters.prix_max) params.append('prix_max',
activeFilters.prix_max);

    container.setAttribute('aria-busy', 'true');
    container.innerHTML = '<p class="loading">Chargement...</
p>';

    fetch(`/menus/filtres?${params.toString()}`, {
        headers: { 'X-Requested-With': 'XMLHttpRequest' }
    })
    .then(response => {
        if (!response.ok) throw new Error('Erreur réseau');
        return response.text();
    })
    .then(html => {
        container.innerHTML = html;
        container.setAttribute('aria-busy', 'false');
    })
    .catch(() => {
        container.innerHTML = '<p class="error"
role="alert">Une erreur est survenue. Veuillez réessayer.</p>';
        container.setAttribute('aria-busy', 'false');
    });
}

```

Le header X-Requested-With: XMLHttpRequest permet au contrôleur PHP de détecter qu'il s'agit d'une requête AJAX et de retourner uniquement le fragment HTML des cartes menus, sans recharger le layout complet.

Calcul de prix en temps réel

Sur le formulaire de commande, le prix total est recalculé instantanément à chaque modification du nombre de personnes.

```

const input = document.getElementById('nb_personnes');
const sousTotalEl = document.getElementById('sous_total');
const totalEl = document.getElementById('total');

const prixMenu = parseInt(input.dataset.prix);

```

```
const minPersonnes = parseInt(input.dataset.min);
const fraisLivraison = 5;
```

```
function updatePrices() {
  let nb = parseInt(input.value) || minPersonnes;
```

```
  if (nb < minPersonnes) {
    nb = minPersonnes;
    input.value = minPersonnes;
  }
```

```
const sousTotal = prixMenu * nb;
const total = sousTotal + fraisLivraison;
```

```
sousTotalEl.textContent = sousTotal.toFixed(2) + ' €';
totalEl.textContent = total.toFixed(2) + ' €';
}
```

```
input.addEventListener('input', updatePrices);
updatePrices()
```

Affichage/masquage du mot de passe

Les formulaires d'authentification intègrent un bouton permettant d'afficher ou de masquer le mot de passe.

```
function togglePassword(inputId, iconId) {
  const passwordInput = document.getElementById(inputId);
  const toggleIcon = document.getElementById(iconId);
```

```
// AFFICHE MDP
```

```
  if (passwordInput.type === 'password') {
    passwordInput.type = 'text';
    toggleIcon.classList.remove('fa-eye');
    toggleIcon.classList.add('fa-eye-slash');
  } else {
    passwordInput.type = 'password';
    toggleIcon.classList.remove('fa-eye-slash');
    toggleIcon.classList.add('fa-eye');
  }
}
```

3.6 Sécurité front-end (CSRF, XSS)

Protection CSRF — Cross-Site Request Forgery

Une attaque CSRF consiste à piéger un utilisateur connecté pour lui faire exécuter une action à son insu sur un site où il est authentifié. Pour s'en prémunir, chaque formulaire POST de l'application intègre un token CSRF unique généré côté serveur.

```
public static function generateCsrfToken(): string
{
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}
```

Le token est injecté dans chaque formulaire via un champ caché :

```
<section class="form-container">
    <form class="contact-form" method="POST" action="/connexion">

        <input type="hidden" name="csrf_token" value="<?=
htmlspecialchars($_SESSION['csrf_token'] ?? '') ?>">
```

"À la réception du formulaire, le contrôleur vérifie que le token soumis correspond au token en session via hash_equals(), une fonction PHP qui garantit une comparaison sécurisée : »

```
public static function verifyCsrfToken(string $token): bool
{
    if (!isset($_SESSION['csrf_token'])) {
        return false;
    }

    return hash_equals($_SESSION['csrf_token'], $token);
}
```

Le token CSRF est présent sur l'ensemble des formulaires POST de l'application : connexion, inscription, commande, avis, contact, et tous les formulaires de l'espace employé et administrateur.

Protection XSS — Cross-Site Scripting

Une attaque XSS consiste à injecter du code JavaScript malveillant dans une page web via les données saisies par les utilisateurs. Pour s'en prémunir, toutes les données affichées dans les templates PHP sont systématiquement échappées via `htmlspecialchars()` avant leur rendu HTML :

```
<label for="nom">Nom <span aria-hidden="true">*</span></label>
<input type="text" id="nom" name="nom"placeholder="Votre nom"
value="<?= htmlspecialchars($old['nom'] ?? '') ?>"
```

4. Développer la partie back-end

4.1 Base de données (MCD, SQL)

Conception

La base de données a été conçue à partir d'un Modèle Conceptuel de Données (MCD) réalisé en amont du développement (voir Annexes). Ce modèle a ensuite été traduit dans le fichier `sql/create.sql`.

La base de données `viteGourmand` est encodée en `utf8mb4` pour garantir la prise en charge complète des caractères spéciaux.

Tables principales

La base comprend 15 tables organisées autour des entités métier principales. Voici la structure des deux tables fondamentales :

```
CREATE TABLE `role` (
  `role_id` INT NOT NULL AUTO_INCREMENT,
  `libelle` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`role_id`),
  UNIQUE KEY `uq_role_libelle` (`libelle`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
```

```
-- TABLE : utilisateur
```

```

CREATE TABLE `utilisateur` (
  `utilisateur_id` INT NOT NULL AUTO_INCREMENT,
  `role_id` INT NOT NULL DEFAULT 1,
  `nom` VARCHAR(100) NOT NULL,
  `prenom` VARCHAR(100) NOT NULL,
  `email` VARCHAR(255) NOT NULL,
  `mot_de_passe` VARCHAR(255) NOT NULL,
  `telephone` VARCHAR(20) NOT NULL,
  `adresse` VARCHAR(255) NOT NULL,
  `code_postal` VARCHAR(10) NOT NULL,
  `ville` VARCHAR(100) NOT NULL,
  `statut` TINYINT(1) NOT NULL DEFAULT 1 COMMENT
  '1=actif, 0=désactivé',
  `date_creation` DATETIME NOT NULL DEFAULT
  CURRENT_TIMESTAMP,
  PRIMARY KEY (`utilisateur_id`),
  UNIQUE KEY `uq_utilisateur_email` (`email`),
  KEY `fk_utilisateur_role` (`role_id`),
  CONSTRAINT `fk_utilisateur_role`
    FOREIGN KEY (`role_id`) REFERENCES `role` (`role_id`)
    ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;

```

"La table utilisateur est liée à la table role par une clé étrangère fk_utilisateur_role, garantissant qu'un utilisateur ne peut être créé qu'avec un rôle existant en base de données.

Accès aux données via PDO

Tous les accès à la base de données transitent par des requêtes préparées PDO.

```

/* Create user */

public function create(Utilisateur $utilisateur): int
{
    try {
        $stmt = $this->pdo->prepare(
            'INSERT INTO utilisateur
              (role_id, nom, prenom, email, mot_de_passe,
telephone, adresse, code_postal, ville)
              VALUES
              (:role_id, :nom, :prenom, :email, :mot_de_passe, :telephone, :adre
sse, :code_postal, :ville)'
        );
        $stmt->execute([

```



```

        ':role_id'      => $utilisateur->getRoleId(),
        ':nom'          => $utilisateur->getNom(),
        ':prenom'       => $utilisateur->getPrenom(),
        ':email'        => $utilisateur->getEmail(),
        ':mot_de_passe' => $utilisateur->getMotDePasse(),
        ':telephone'    => $utilisateur->getTelephone(),
        ':adresse'      => $utilisateur->getAdresse(),
        ':code_postal'  => $utilisateur->getCodePostal(),
        ':ville'        => $utilisateur->getVille(),
    ];

    return (int) $this->pdo->lastInsertId();

} catch (PDOException $e) {
    error_log('[UtilisateurRepository::create] ' . $e-
>getMessage());
    throw new \RuntimeException('Erreur lors de la
création de l\'utilisateur.');
```

Chaque paramètre est passé via un tableau de valeurs nommées, jamais concaténé dans la chaîne SQL. PDO se charge d'échapper automatiquement les valeurs, rendant toute injection SQL impossible.

4.2 Architecture MVC PHP POO

L'application repose sur une architecture MVC (Modèle - Vue - Contrôleur) développée entièrement en PHP 8.3 orienté objet. Cette architecture a été construite de zéro et organise le code en couches aux responsabilités distinctes, le Modèle est divisé en deux couches distinctes : les **Entités** qui représentent les objets métier, et les **Repositories** qui gèrent l'accès aux données.

Structure du projet

```

src/
├─ Controller/  = Un contrôleur par fonctionnalité
├─ Entity/      = Classes métier (Utilisateur, Menu, Commande...)
├─ Repository/  = Accès BDD via PDO
├─ Service/     = Logique métier (Auth, Mail, Price, Mongo)
└─ Core/        = Router, Database, Session, Controller de base
templates/      = Vues PHP
public/         = Point d'entrée unique (index.php)
```

Le Router

Toutes les requêtes HTTP transitent par un point d'entrée unique **public/index.php**, qui instancie le Router. Celui-ci détermine le contrôleur et l'action à exécuter. Chaque route est associée à un rôle obligatoire, le Router vérifie les droits d'accès avant d'instancier le contrôleur :

```
public function dispatch(): void
{
    $method = $_SERVER['REQUEST_METHOD'];
    $uri     = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
    $uri     = rtrim($uri, '/') ?: '/';

    foreach ($this->routes as $route) {
        if ($route['method'] === $method && $route['path'] ===
$uri) {

            // Vérif role

            $this->checkAccess($route['role']);

            // Instance

            $controllerClass = 'Controller\\' . $route['controller'];
            $action = $route['action'];

            if (!class_exists($controllerClass)) {
                $this->abort(500, "Contrôleur introuvable :
{$controllerClass}");
                return;
            }

            $controller = new $controllerClass();

            if (!method_exists($controller, $action)) {
                $this->abort(500, "Action introuvable :
{$action}");
                return;
            }

            $controller->$action();
            return;
        }
    }
}
```

Un exemple de contrôleur : HomeController

Chaque contrôleur hérite d'une classe de base Controller et suit le même schéma : injection des repositories nécessaires dans le constructeur, récupération des données, puis rendu de la vue avec la méthode render() :

```
class HomeController extends Controller
{
    private AvisRepository $avisRepository;
    private HoraireRepository $horaireRepository;

    public function __construct()
    {
        $this->avisRepository = new AvisRepository();
        $this->horaireRepository = new HoraireRepository();
    }

    public function index(): void
    {
        $avis = $this->avisRepository->findValides();
        $horaire = $this->horaireRepository->getTexte();

        $this->render('home/index', [
            'avis' => $avis,
            'horaire' => $horaire,
            'error' => Session::getFlash('error'),
            'success' => Session::getFlash('success'),
        ]);
    }
}
```

Le contrôleur orchestre les appels aux repositories et aux services, puis transmet les données à la vue.

4.3 Composants d'accès aux données (PDO + MongoDB)

Accès aux données relationnelles : PDO

Tous les accès à la base de données MySQL passent par la classe AbstractRepository qui centralise la connexion PDO. Chaque repository hérite de cette classe.

Accès aux données non relationnelles : MongoDB

MongoDB est utilisé pour stocker et analyser les statistiques de commandes terminées. Les statistiques sont des données d'analyse qui n'ont pas de contraintes relationnelles de MySQL.

Enregistrement d'une commande terminée

Lorsqu'une commande passe au statut "terminée", un document est inséré dans la collection MongoDB :

```
public function enregistrerCommande(
    int $commandeId,
    int $menuId,
    string $menuTitre,
    string $dateCommande,
    int $nombrePersonnes,
    float $prixTotal
): void {
    try {
        $this->collection->insertOne([
            'commande_id' => $commandeId,
            'menu_id' => $menuId,
            'menu_titre' => $menuTitre,
            'date_commande' => $dateCommande,
            'nombre_personnes' => $nombrePersonnes,
            'prix_total' => $prixTotal,
            'statut' => 'terminee',
            'created_at' => new
\MongoDB\BSON\UTCDateTime(time() * 1000),
        ]);
    }
```

4.4 Composants métier (Services, Entités)

Les Entités, encapsulation des données

Les entités représentent les objets métier de l'application. Chaque entité encapsule ses propriétés en private et n'expose ses données qu'à travers des getters et setters, conformément au principe d'encapsulation de la POO.

L'entité Commande illustre cette approche, en définissant les statuts possibles comme constantes de classe, garantissant qu'aucune valeur invalide ne peut être assignée :

```
class Commande
{
    public const STATUT_EN_ATTENTE = 'en_attente';
    public const STATUT_ACCEPTEE = 'acceptee';
    public const STATUT_EN_PREPARATION = 'en_preparation';
}
```

```

public const STATUT_EN_COURS_LIVRAISON =
'en_cours_livraison';
public const STATUT_LIVREE = 'livree';
public const STATUT_EN_ATTENTE_RETOUR_MATERIEL =
'en_attente_retour_materiel';
public const STATUT_TERMINEE = 'terminee';
public const STATUT_ANNULEE = 'annulee';

```

```

public function setStatut(string $statut): void
{
    if (!in_array($statut, self::STATUTS_VALIDES)) {
        throw new \InvalidArgumentException("Statut invalide :
{$statut}");
    }
    $this->statut = $statut;
}

```

La méthode `estModifiable()` encapsule une règle métier directement dans l'entité, une commande n'est modifiable que si elle est en attente :

```

public function estModifiable(): bool
{
    return $this->statut === self::STATUT_EN_ATTENTE;
}

```

Les Services, logique métier

La logique métier complexe est isolée dans des classes `Service`. L'application dispose de quatre services principaux : `AuthService`, `MailService`, `PriceService` et `MongoService`.

Le `PriceService` illustre cette séparation, il centralise toutes les règles de calcul des prix sans aucun accès BDD :

```

public function calculerTotal(
    Menu $menu,
    int $nombrePersonnes,
    string $codePostal,
    float $distanceKm = 0
): array {
    $prixMenu = $this->calculerPrixMenu($menu,
        $nombrePersonnes);
    $prixLivraison = $this->calculerFraisLivraison($codePostal, $distanceKm);
    $prixTotal = round($prixMenu + $prixLivraison, 2);
}

```

```
return [
    'prix_menu'      => $prixMenu,
    'prix_livraison' => $prixLivraison,
    'prix_total'     => $prixTotal,
];
```

Les règles métier sont définies comme constantes de classe, ce qui les rend facilement modifiables sans toucher à la logique :

```
class PriceService
{
    private const FRAIS_LIVRAISON_BASE = 5.00;
    private const PRIX_PAR_KM         = 0.59;
    private const CODE_POSTAL_BORDEAUX = '33000';
```

4.5 Sécurité back-end, protection des données et conformité RGPD

Hashage des mots de passe

Les mots de passe ne sont jamais stockés en clair en base de données. Lors de l'inscription et de la réinitialisation du mot de passe, la fonction PHP `password_hash()` avec l'algorithme `PASSWORD_DEFAULT` (bcrypt) produit un hash irréversible :

```
$utilisateur->setMotDePasse(password_hash($data['mot_de_passe'],
PASSWORD_DEFAULT));
```

La vérification lors de la connexion utilise `password_verify()` qui compare le mot de passe saisi avec le hash stocké sans le déchiffrer.

Contrôle d'accès par rôle

Chaque route de l'application est associée à un rôle minimum requis. Le Router vérifie les droits de l'utilisateur connecté avant d'instancier le contrôleur grâce à un système de hiérarchie de rôles.

```
private function checkAccess(?string $requiredRole): void
{
    if ($requiredRole === null) {
        return;
    }

    if (!Session::isLoggedIn()) {
        Session::setFlash('error', 'Vous devez être connecté
pour accéder à cette page.');
```

```

        Session::set('redirect_after_login',
$_SERVER['REQUEST_URI']);
        header('Location: /connexion');
        exit;
    }

    $userRole = Session::getUserRole();

    // Hiérarchie

    $hierarchy = ['utilisateur' => 1, 'employe' => 2,
'administrateur' => 3];

    $userLevel = $hierarchy[$userRole] ?? 0;
    $requiredLevel = $hierarchy[$requiredRole] ?? 0;

    if ($userLevel < $requiredLevel) {
        $this->abort(403);
    }
}

```

Ce système garantit qu'un utilisateur ne peut jamais accéder à une route qui dépasse son niveau de droits : un employé ne peut pas accéder aux routes administrateur, et un visiteur non connecté est redirigé vers la page de connexion.

Protection contre les injections SQL

Comme détaillé en section 4.1, tous les accès à la base de données utilisent exclusivement des requêtes préparées PDO avec paramètres nommés.

Variables d'environnement

Les données sensibles (mots de passe BDD, clés SMTP, identifiants MongoDB) sont stockés dans un fichier `.env` jamais commité dans Git. Le fichier `.env.example` fourni dans le repository documente les variables nécessaires sans exposer les valeurs.

Références et conformité

Les mesures de sécurité mises en place s'appuient sur les recommandations de l'OWASP Top 10, qui liste les dix vulnérabilités web les plus critiques. Les principales menaces sont l'injection SQL, les failles d'authentification, et les mauvaises configurations de sécurité

Concernant le RGPD et les recommandations de la CNIL, les données personnelles des utilisateurs (nom, prénom, email, adresse, téléphone) sont collectées uniquement dans le cadre de la

commande, stockées de façon sécurisée, et accessibles uniquement à l'utilisateur concerné et aux administrateurs.

5. Tests et jeu d'essai

Tests unitaires PHPUnit

La logique du **PriceService** est couverte par des tests unitaires PHPUnit. Ces tests vérifient automatiquement que les calculs de prix donnent les résultats attendus dans tous les cas: livraison à Bordeaux, hors Bordeaux, avec distance kilométrique.

Les tests couvrent également les entités **Commande** et **Avis**, ils vérifient que les règles de validation métier lèvent bien les exceptions attendues en cas de données invalides.

La suite de tests compte 23 tests et 29 assertions validés :

```
Runtime:      PHP 8.3.30
Configuration: /var/www/html/phpunit.xml

.....                                     23 / 23 (100%)

Time: 00:00.014, Memory: 14.00 MB

Avis
✓ Note valide
✓ Note minimum
✓ Note trop elevee leve exception
✓ Note zero leve exception
✓ Commentaire valide
✓ Commentaire vide leve exception
✓ Statut invalide leve exception

Commande
✓ Est modifiable en attente
✓ Est modifiable acceptee retourne false
✓ Est modifiable terminee retourne false
✓ Nombre personnes valide
✓ Nombre personnes zero leve exception
✓ Statut libelle en attente
✓ Statut libelle terminee
✓ Statut invalide leve exception

Price Service
✓ Prix menu normal
✓ Prix menu minimum
✓ Prix menu sous minimum leve exception
✓ Livraison bordeaux
✓ Livraison hors bordeaux avec km
✓ Total bordeaux
✓ Total hors bordeaux
✓ Retour tableau avec cles correctes

OK (23 tests, 29 assertions)
```


Voici un extrait représentatif des tests du PriceService :

```
public function testTotalBordeaux(): void
{
    $result = $this->priceService->calculerTotal($this->menu,
4, '33000');
    $this->assertEquals(100.00, $result['prix_menu']);
    $this->assertEquals(5.00, $result['prix_livraison']);
    $this->assertEquals(105.00, $result['prix_total']);
}
```

```
public function testPrixMenuSousMinimumLeveException(): void
{
    $this->expectException(\InvalidArgumentException::class);
    $this->priceService->calculerPrixMenu($this->menu, 1); //
minimum est 2
}
```

Le premier test vérifie le calcul complet d'une commande à Bordeaux : 4 personnes à 25€ avec 5€ de frais de livraison. Le second vérifie que le service lève bien une exception si le nombre de personnes est inférieur au minimum requis par le menu.

Jeu d'essai

Compte utilisé : user@vitegourmand.fr

Menu testé : Le Réception : 36€/personne, minimum 2 personnes

Date de prestation : 18/06/2026

Livraison : Bordeaux (33000)

Les validations front-end (select minimum personnes, datepicker) empêchent la saisie de valeurs invalides avant même la soumission du formulaire. Une validation serveur complémentaire est en place dans le Controller pour les requêtes directes.

commande réussie à Bordeaux

Action	Données en entrée	Résultat attendu	Résultat obtenu
Sélection du menu	"Le Réception"	Fiche menu complète	Affiché
Calcul du prix	4 personnes : 36€/pers code postal 33000	144€ + 5€ livraison = 149€	149 €
Validation commande	Date 18/06/2026 + adresse complète + heure livraison	Commande créée en BDD, statut "en_attente", numéro généré	Créée
Email confirmation	Validation du formulaire	Email de confirmation envoyé au client	Email reçu
Suivi commande client	Mes commandes	Commande visible avec statut "en_attente" et détails	Affiché
Changement statut employé	Statut = livrée depuis espace employé	Statut mis à jour en BDD, historique suivi enregistré	Mis à jour
Suivi côté client	Mes commandes	Statut "livrée" côté client	Affiché

Cas erreur, validations front-end

Action	Données en entrée	Résultat attendu	Résultat obtenu
Nombre de personnes insuffisant	Tentative saisie 1 personne (minimum 2)	Saisie impossible car select bloqué côté client	Select bloqué
Date trop proche	Tentative saisie date < 6 jours	Saisie impossible car datepicker bloqué côté client	Datepicker bloqué

Cas annulation

Action	Données en entrée	Résultat attendu	Résultat obtenu
Annulation commande en attente	Clic "Annuler" : statut "en_attente"	Statut = "annulée", confirmation affichée	Annulée

Captures du jeu d'essai

Choix du menu « Le Réception »

Gastronomie

"Le Réception"

Un menu parfait pour accompagner vos réceptions et événements.

Sashimi de saumon, betteraves et fleurs de sureau

Fondant de bœuf et son écrasé de panais

Entremet pistache et sa crème vanille de Madagascar

Ce menu doit être commandé **6 jours minimum** avant la date de la prestation.

Tous nos plats sont fabriqués à base de produits frais.

Ils doivent être conservés au réfrigérateur et peuvent être congelés.

COMMANDER

Panier

Votre commande

Le Réception

2 personnes minimum — 36,00€ / pers.

Nombre de personnes

4

Minimum 2 personnes.

Valider

Total

Sous-total menu :

144,00 €

Frais de livraison :

5,00 €

Total :

149,00 €

Le paiement s'effectue au moment de la livraison. Nous acceptons :

Commande
VG-20260002

Le Réception



Informations

Adresse de livraison

6 rue Lucie Aubrac

Ville

Bordeaux

Nombre de personnes

4

Date de prestation

18 / 06 / 2026

Heure de livraison

13 : 13

Prix total

149,00 €

MODIFIER

ANNULER

SUIVRE

Changement du statut coté employé

BIENVENUE DANS L'ESPACE EMPLOYÉ

ACCUEIL EMPLOYÉ

GESTION DES COMMANDES

⚠ Contacter le client avant d'annuler sa commande.

Commandes

Laurent Dubois
N° : VG-20260002
Menu : Le Réception
Personnes : 4
Total : 149,00 €
Téléphone : 07 55 44 33 28
Adresse : 6 rue Lucie Aubrac, Bordeaux
Date : 18/06/2026 à 13:13

Livree

Statut changé coté client

Commande VG-20260002

Informations

Adresse de livraison
6 rue Lucie Aubrac

Ville

Heure de livraison
13 : 13

Prix total
149,00 €

Le Réception



SUIVI

Votre commande est actuellement

LIVRÉE

MODIFICATION IMPOSSIBLE

Votre commande a déjà été prise en charge.

SUIVRE

Cas d'annulation de la commande

BIENVENUE DANS L'ESPACE EMPLOYÉ

ACCUEIL EMPLOYÉ

GESTION
DES
COMMANDES

⚠ Contacter le client avant d'annuler sa commande.

Commandes

Laurent Dubois
N° : VG-20260002
Menu : Le Réception
Personnes : 4
Total : 149,00 €
Téléphone : 07 55 44 33 28
Adresse : 6 rue Lucie Aubrac, Bordeaux
Date : 18/06/2026 à 13:13

Motif d'annulation :
fermeture exceptionnelle

Annulée

6. Veille technologique et sécurité

6.1 Veille sécurité : OWASP et hashage des mots de passe

Lors du développement du système d'authentification, je me suis interrogé sur le choix de l'algorithme de hashage à utiliser pour stocker les mots de passe des utilisateurs. J'ai consulté l'OWASP Password Storage Cheat Sheet, document de référence de la fondation OWASP (Open Web Application Security Project) qui liste les bonnes pratiques de sécurité web.

Extrait de la source en anglais

(Source : https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)

"Because hashing is a one-way function (it is impossible to "decrypt" a hash and obtain the original plaintext value), it is the most appropriate approach for password validation. Even if an attacker obtains the hashed password, they cannot use it to log in as the victim.

For legacy systems using bcrypt, use a work factor of 10 or more and with a password limit of 72 bytes."

Traduction

« Le hashage est une fonction à sens unique (il est impossible de retrouver le mot de passe original à partir du hash stocké). C'est donc l'approche la plus appropriée pour la validation des mots de passe. Même si un attaquant accède aux hashes stockés en base de données, il ne peut pas les utiliser directement pour se connecter.

Pour les systèmes utilisant bcrypt, utilisez un facteur de 10 caractères ou plus avec une limite de mot de passe de 72 octets." »

7. Déploiement

7.1 Conteneurisation avec Docker

L'application est entièrement conteneurisée via Docker et Docker Compose. Cette approche garantit que l'environnement d'exécution est identique en développement et en production.

L'architecture Docker de production comprend quatre conteneurs :

Conteneur	Image	Rôle
vg_app	PHP 8.3 custom	Exécution du code PHP
vg_nginx	nginx:alpine	Serveur web, reverse proxy
vg_mysql	mysql:8.0	Base de données relationnelle
vg_mongo	mongo:7.0	Base de données non relationnelle

7.2 Différences entre développement et production

Le fichier `docker-compose.prod.yml` diffère du fichier de développement sur plusieurs points importants :

- PhpMyAdmin est absent en production
- Les ports MySQL et MongoDB ne sont pas exposés
- Restart: always est activé sur tous les conteneurs
- Les volumes `vg_mysql_data` et `vg_mongo_data` assurent la persistance des données entre les redémarrages

7.3 Sécurité du déploiement

Toutes les données sensibles: mots de passe BDD, identifiants MongoDB, clés SMTP, sont externalisées dans un fichier `.env` non commité dans Git. Le fichier `.env.example` fourni les variables nécessaires sans exposer les valeurs réelles.

7.4 Procédure de déploiement sur VPS

1. Cloner le projet

```
git clone https://github.com/thomas-bruston/viteGourmandV2.git cd viteGourmandV2
```

2. Configurer les variables d'environnement

```
cp .env.example .env
```

```
# Éditer .env : APP_ENV=production, APP_URL=https://vitegourmand.tech
```

3. Lancer les conteneurs en production

```
docker compose -f docker-compose.prod.yml up -d --build
```

4. Installer les dépendances

```
docker compose exec app composer install --no-dev --optimize-autoloader
```

5. Initialiser la base de données

```
docker compose exec -T mysql mysql -u root -prootpassword < sql/create.sql
```

```
docker compose exec -T mysql mysql -u root -prootpassword --default-character-set=utf8mb4  
viteGourmand < sql/seed.sql
```

6. Configurer HTTPS avec Let's Encrypt

```
sudo apt install certbot -y
```

```
docker compose -f docker-compose.prod.yml stop nginx
```

```
sudo certbot certonly --standalone -d vitegourmand.tech -d www.vitegourmand.tech
```

```
sudo cp /etc/letsencrypt/live/vitegourmand.tech/fullchain.pem docker/nginx/ssl/
```

```
sudo cp /etc/letsencrypt/live/vitegourmand.tech/privkey.pem docker/nginx/ssl/
```

```
docker compose -f docker-compose.prod.yml start nginx
```

7. Vérifier que tout fonctionne

```
docker compose -f docker-compose.prod.yml ps
```

<https://vitegourmand.tech>

8. Conclusion

Ce projet de quatre mois a représenté un défi technique et personnel considérable. Venant d'un parcours dans le management avec une expérience en design graphique, j'étais bien plus à l'aise sur les parties maquettage, HTML et CSS que sur le développement back-end en PHP. C'est précisément cette partie qui s'est révélée la plus difficile, comprendre et maîtriser la programmation orientée objet, l'architecture MVC, les accès aux bases de données via PDO, et la gestion de la sécurité.

C'est donc avec fierté que je présente aujourd'hui cette application fullstack complète et fonctionnelle. Avoir réussi à concevoir et implémenter une architecture MVC en PHP, sans framework, avec des repositories, des services métier, une double base de données SQL et NoSQL, et un système de sécurité complet, représente pour moi l'accomplissement majeur de ce projet. Ayant découvert Angular lors de mon parcours de formation, j'ai retrouvé dans la conception de ce projet des principes architecturaux similaires; séparation des services, du routing et de la logique métier; que j'ai appliqués nativement en PHP orienté objet.

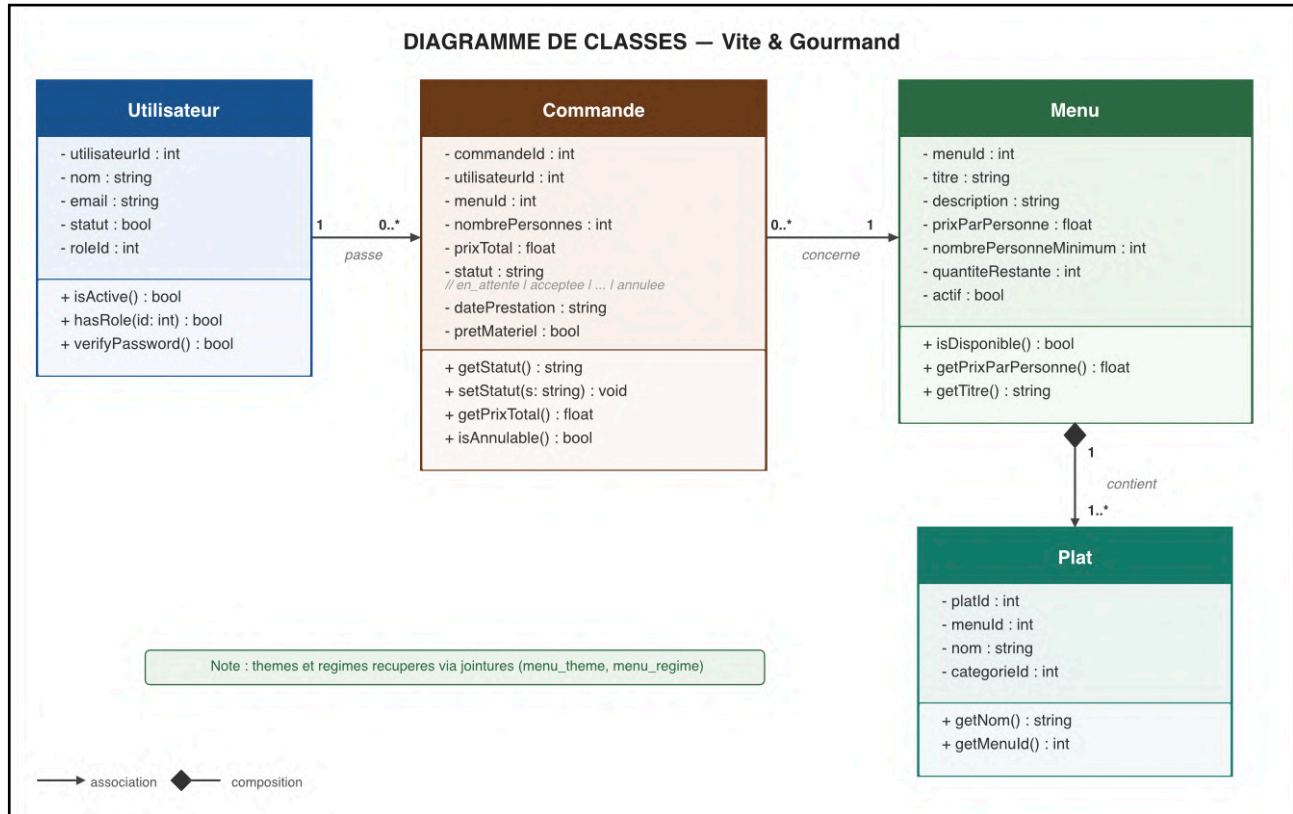
Ce projet a confirmé ma vocation pour le développement web et ouvert une opportunité professionnelle concrète. Je me consacre désormais à l'apprentissage d'Angular pour intégrer Tachycard, une startup du milieu médical développant une application web pour les cardiologues et les patients équipés de pacemakers, entreprise au sein de laquelle j'ai effectué une immersion professionnelle avant ma formation.

9. Annexes

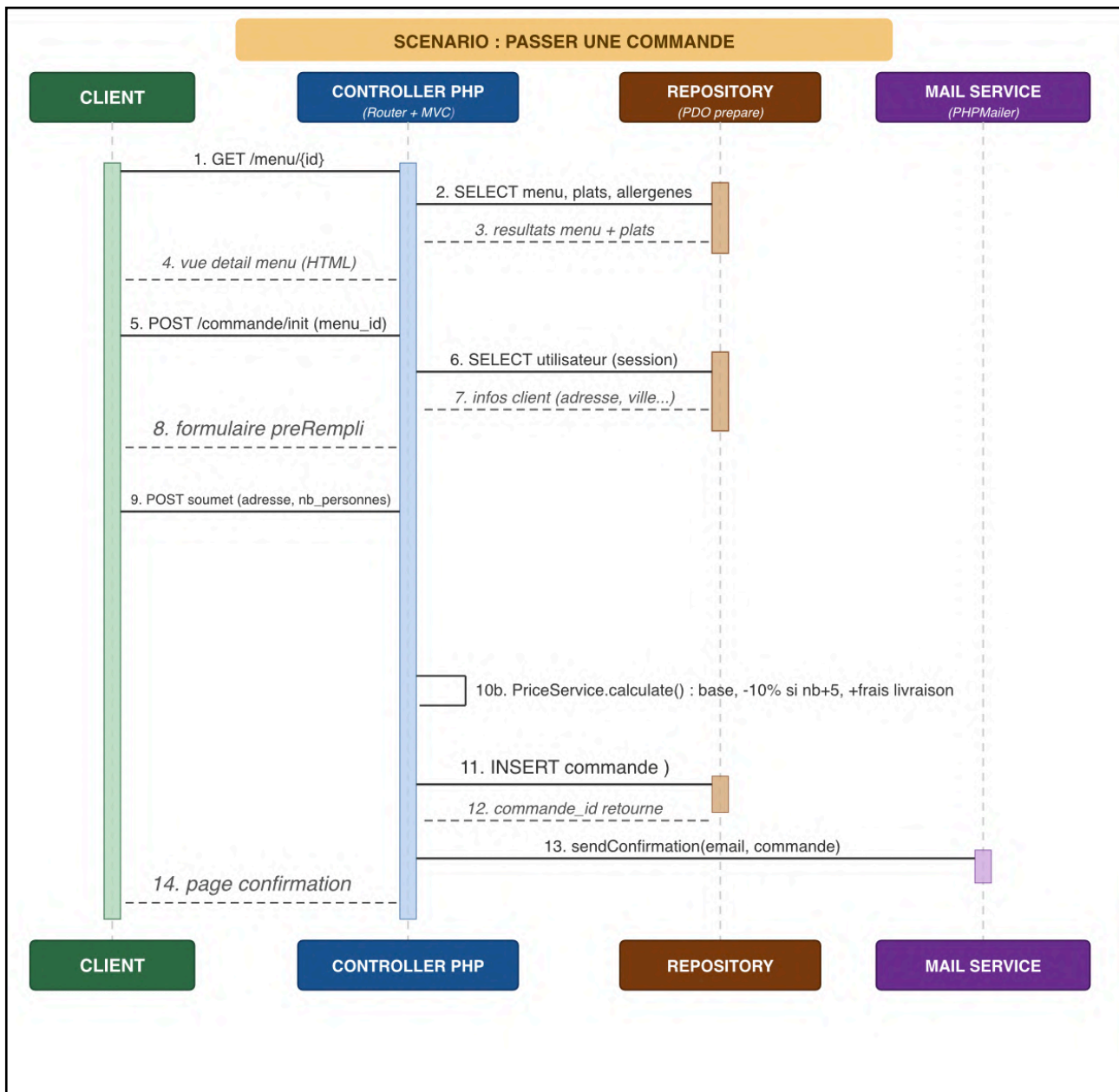
Liste des annexes

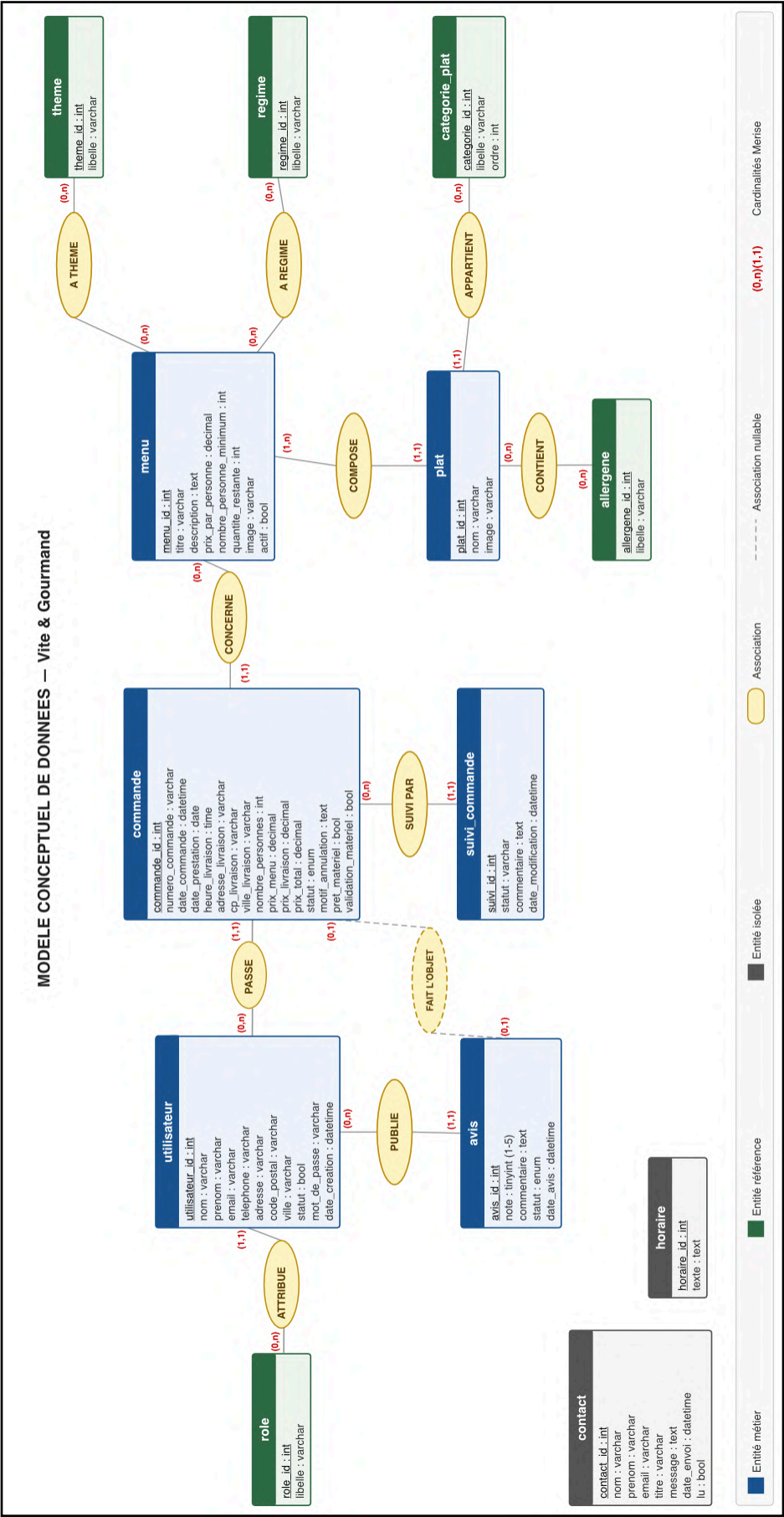
- Annexe 1 Diagramme de classes : entités PHP
- Annexe 2 Diagramme de séquences : parcours commande
- Annexe 3 MCD : Modèle Conceptuel de Données
- Annexe 4 Captures d'écran Lighthouse : scores accessibilité
- Annexe 5 Captures d'écran de l'application
- Annexe 6 Lien GitHub du projet

Annexe 1 : Diagramme de classes : entités PHP



Annexe 2 : Diagramme de séquences : parcours commande





Annexe 4 : Captures d'écran Lighthouse, scores accessibilité

Accueil

Vite et Gourmand

Fondé en 2008 par trois amis traiteurs passionnés, notre établissement vous propose une cuisine de qualité.

Nous avons à cœur de favoriser le travail de produits locaux, frais, et issus de circuits courts principalement français, en collaborant étroitement avec les producteurs de la région de Bordeaux.

Nos plats sont non seulement délicieux mais aussi respectueux de l'environnement et de l'économie locale. En privilégiant les produits de saison, nous vous offrons des saveurs authentiques qui reflètent les richesses de notre région.

Cette démarche éthique nous permet de soutenir les agriculteurs et artisans locaux tout en vous proposant des menus gourmands, créatifs, et diversifiés.

Lighthouse Scores:

- Performance: 92
- Accessibility: 95
- Best Practices: 96
- SEO: 100

Performance Score: 92

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator.](#)

Connexion

SE CONNECTER

Votre E-mail

E-mail

Votre mot de passe

Mot de passe

Mot de passe oublié ?

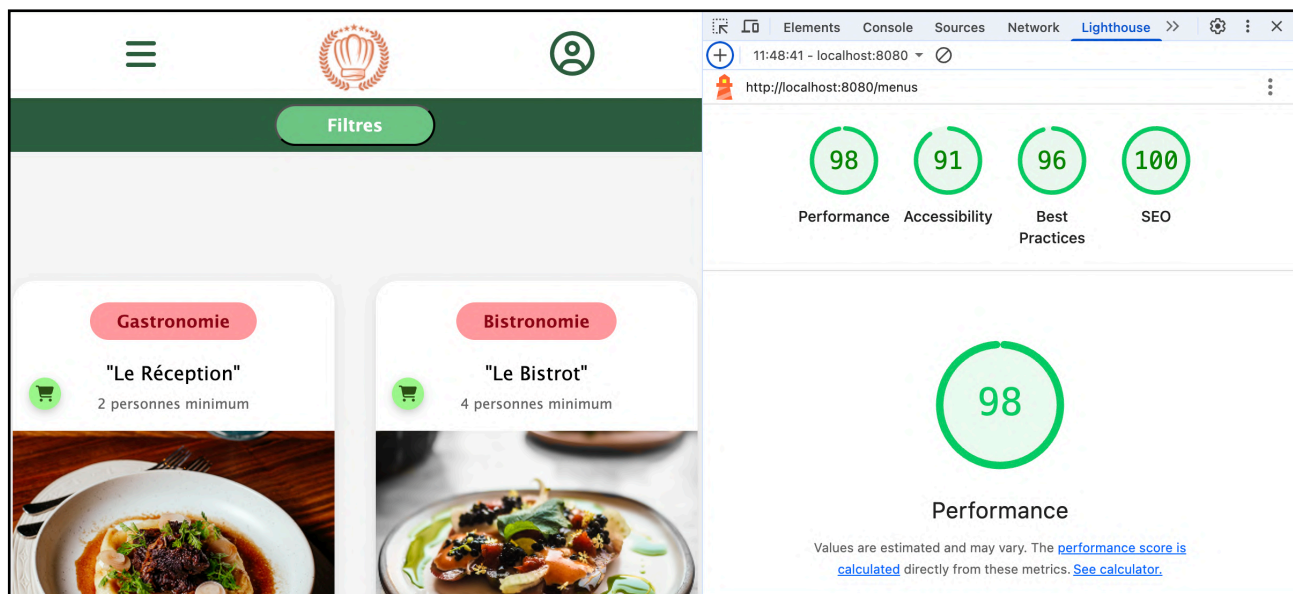
Lighthouse Scores:

- Performance: 99
- Accessibility: 95
- Best Practices: 96
- SEO: 100

Performance Score: 99

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator.](#)

Menu detail



Annexe 5 : Captures d'écran de l'application

Accueil



Laisser un avis

LAISSER UN AVIS

Dans le cadre de notre politique de respect, nous vous demandons de rester courtois lors de la rédaction de votre avis, afin de garantir le respect envers nos employés et nos utilisateurs. Tout avis ne respectant pas ces conditions sera supprimé. Merci.

Votre note *

★★★★★

Votre commentaire *

Prestation parfaite!!! Nous nous sommes régalés.

Entre 10 et 1000 caractères.

DÉPOSER MON AVIS

Menu avec fenêtre filtre ouverte

[Accueil](#)[Menus](#)[Contact](#)[Connexion](#)

Gastronomie

"Le Réception"

2 personnes minimum



36€

Régime : Classique

Cuisine du monde

THÈME

Bistronomie

Cuisine du monde

Événements

Gastronomie

Végétarien

RÉGIME ALIMENTAIRE

Classique

Sans porc

Végétarien

PRIX MAXIMUM PAR PERSONNE

Jusqu'à 70€

NOMBRE DE PERSONNES MINIMUM

2 pers.

4 pers.

6 pers.

8 pers.

10 pers.

Valider

Réinitialiser

Bistronomie

"Le Dinatoire"

6 personnes minimum

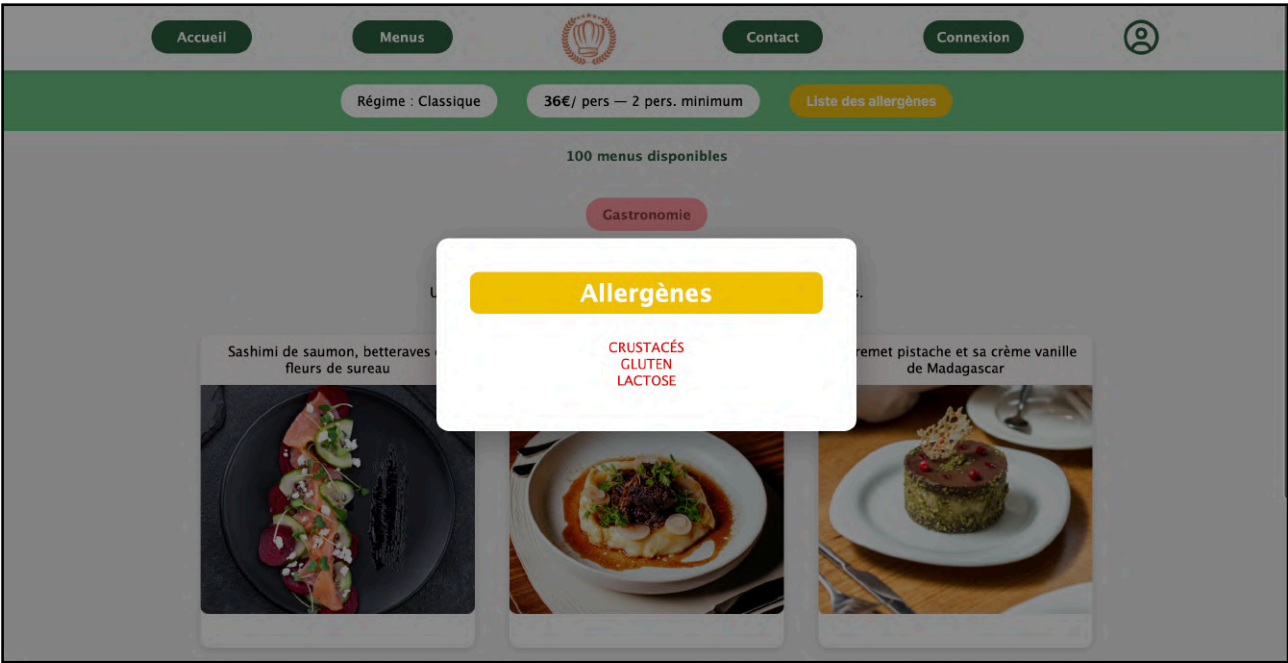


20€

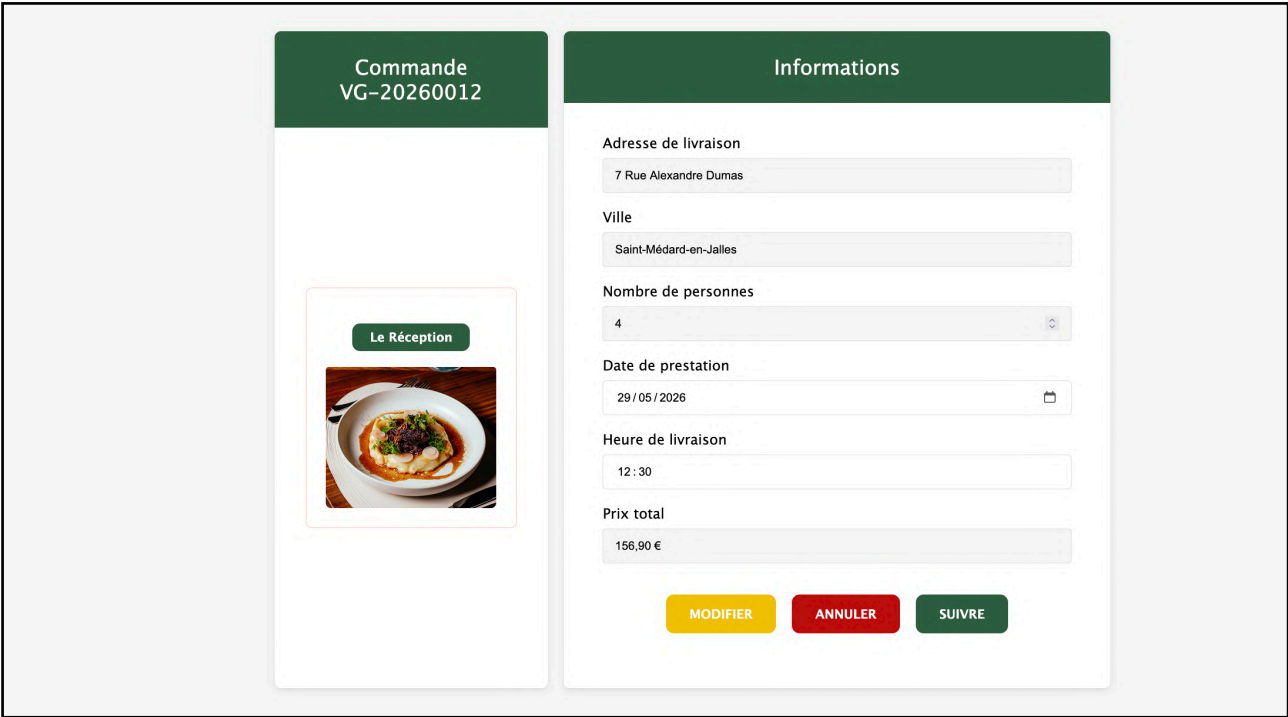
Régime : Classique

Cuisine du monde

Menu details avec fenêtre allergènes ouverte



Mes commandes



Panier

Votre commande

Le Réception



2 personnes minimum — 36,00€ / pers.

Nombre de personnes

4

Minimum 2 personnes.

Total

Sous-total menu :144,00 €

Frais de livraison :12,90 €

Total :156,90 €

Le paiement s'effectue au moment de la livraison. Nous acceptons :



VOS INFORMATIONS POUR LA LIVRAISON

Nom

Dubois

Prénom

Laurent

E-mail

user@vitegourmand.fr

Téléphone

07 55 44 33 28

Adresse de livraison *

7 Rue Alexandre Dumas

Code postal *

33160

Ville *

Saint-Médard-en-Jalles

Date de prestation *

29 / 05 / 2026

Minimum 6 jours à l'avance.

Heure de livraison *

12 : 30

Entre 09h00 et 20h00.

Souhaitez-vous du matériel (plats, équipements) ?

Le matériel doit être restitué sous 10 jours ouvrés. Des frais de 600€ s'appliquent en cas de non-restitution.

VALIDER MA COMMANDE

Espace employé



Gestion des commandes



Ajout d'un menu

Créer un menu

ACCUEIL EMPLOYÉ

Thème

Bistronomie

Titre

Régime

Classique

Prix / pers. (€)

25

Personnes min.

2

Description

Image du menu

Parcourir... Aucun fichier sélectionné.

Plats du menu

Entrée

Nom du plat

Photo du plat

Parcourir... Aucun fichier sélectionné.

☐ Arachides

☐ Crustacés

☐ Gluten

☐ Lactose

☐ Moutarde

☐ Œuf

☐ Soja

Nom du plat

Photo du plat

Parcourir...

Aucun fichier sélectionné.

☐ Arachides

☐ Lactose

☐ Soja

☐ Crustacés

☐ Moutarde

☐ Gluten

☐ Œuf

Dessert

Nom du plat

Photo du plat

Parcourir...

Aucun fichier sélectionné.

☐ Arachides

☐ Lactose

☐ Soja

☐ Crustacés

☐ Moutarde

☐ Gluten

☐ Œuf

Allergènes du menu

☐ Arachides

☐ Lactose

☐ Soja

☐ Crustacés

☐ Moutarde

☐ Gluten

☐ Œuf

CRÉER LE MENU

Dashboard administrateur



ACCUEIL ADMIN

AJOUTER UN EMPLOYÉ

CRÉER UN COMPTE EMPLOYÉ

L'employé recevra un email avec son identifiant de connexion. Communiquez-lui son mot de passe directement.

Nom *

Son nom

Prénom *

Son prénom

E-mail *

Son adresse email

Téléphone

06 12 34 56 78

Adresse

Adresse complète

Mot de passe *

Mot de passe

10 caractères minimum — 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial.

Confirmer le mot de passe *

Confirmation

CRÉER

Statistiques

